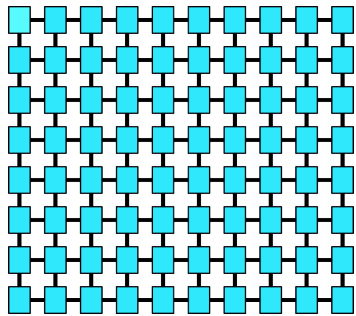


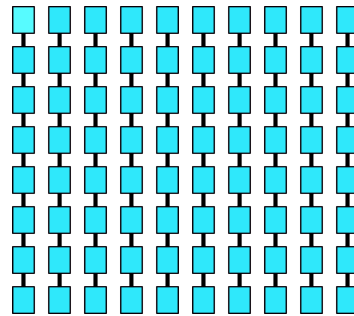
Your input here is welcome!

Lagrangian (Dual) Decomposition



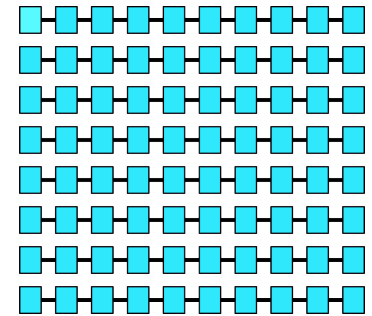
$$\langle \theta, \delta(x) \rangle$$

=



$$\langle \theta^c, \delta(x) \rangle$$

+

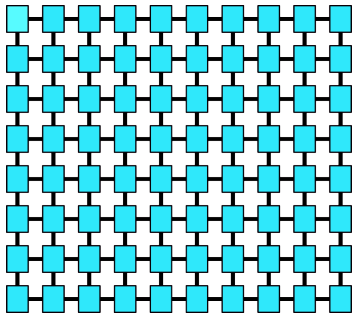


$$\langle \theta^r, \delta(x) \rangle$$

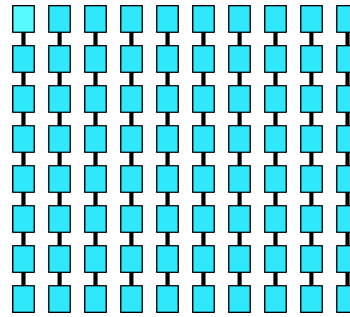
=

+

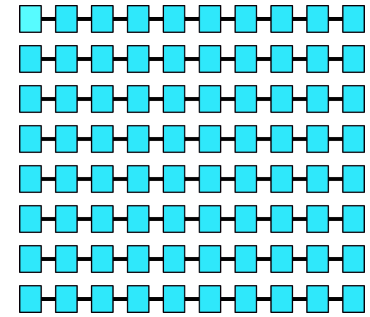
Lagrangian (Dual) Decomposition



=



+



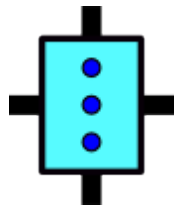
$$\langle \theta, \delta(x) \rangle$$

=

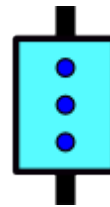
$$\langle \theta^c, \delta(x) \rangle$$

+

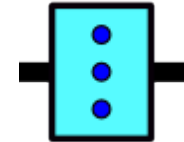
$$\langle \theta^r, \delta(x) \rangle$$



=



+



$$\theta_v(x_v)$$

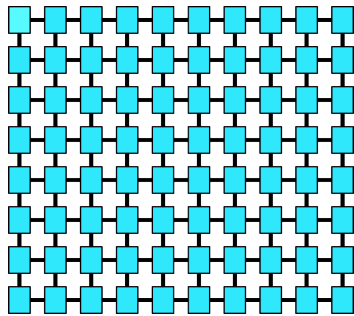
=

$$\frac{\theta_v(x_v)}{2} + \lambda_v(x_v)$$

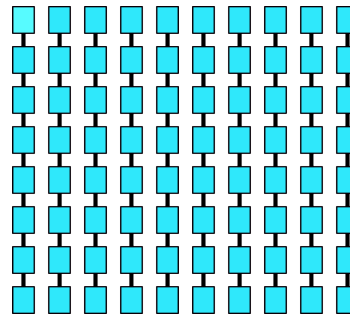
+

$$\frac{\theta_v(x_v)}{2} - \lambda_v(x_v)$$

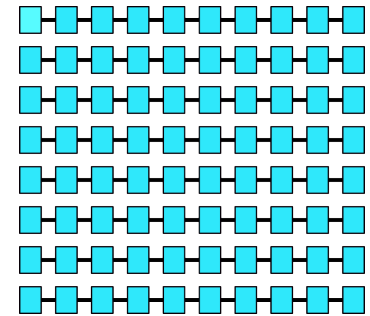
Lagrangian (Dual) Decomposition



=



+



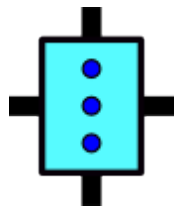
$$\langle \theta, \delta(x) \rangle$$

=

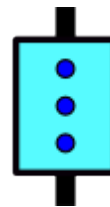
$$\langle \theta^c, \delta(x) \rangle$$

+

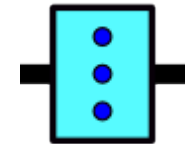
$$\langle \theta^r, \delta(x) \rangle$$



=



+



$$\theta_v(x_v)$$

=

$$\frac{\theta_v(x_v)}{2} + \lambda_v(x_v)$$

+

$$\frac{\theta_v(x_v)}{2} - \lambda_v(x_v)$$

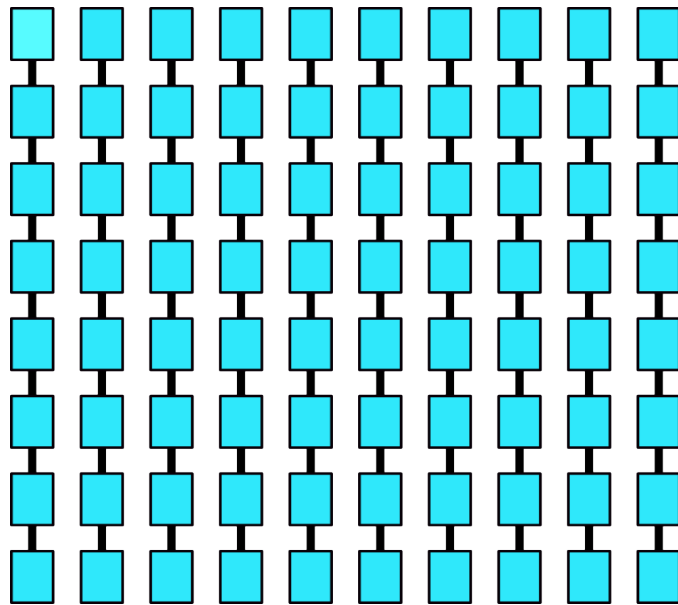
$$\min_{x \in \mathcal{X}} \langle \theta, \delta(x) \rangle$$

$\forall \lambda$
 $\geq$

$$\min_{x^c \in \mathcal{X}^c} \langle \theta_{[\lambda]}^c, \delta(x^c) \rangle$$

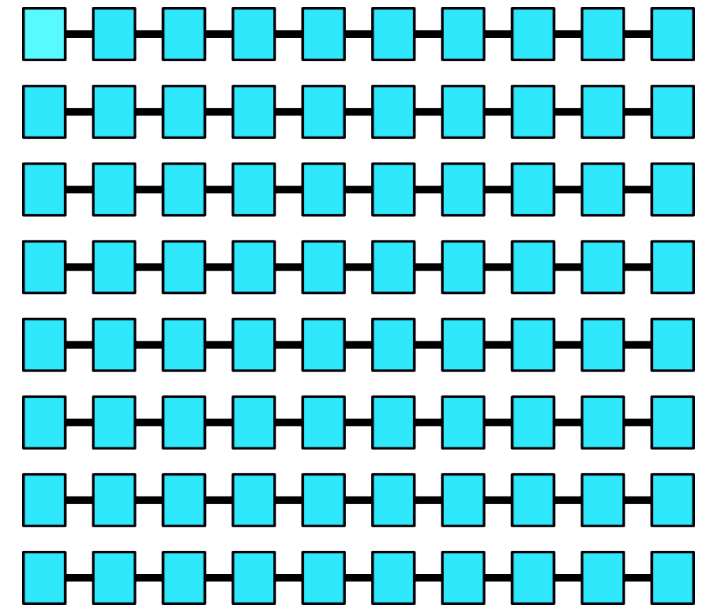
+

$$\min_{x^r \in \mathcal{X}^r} \langle \theta_{[\lambda]}^r, \delta(x^r) \rangle$$



$$x_{i,j}^{c*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

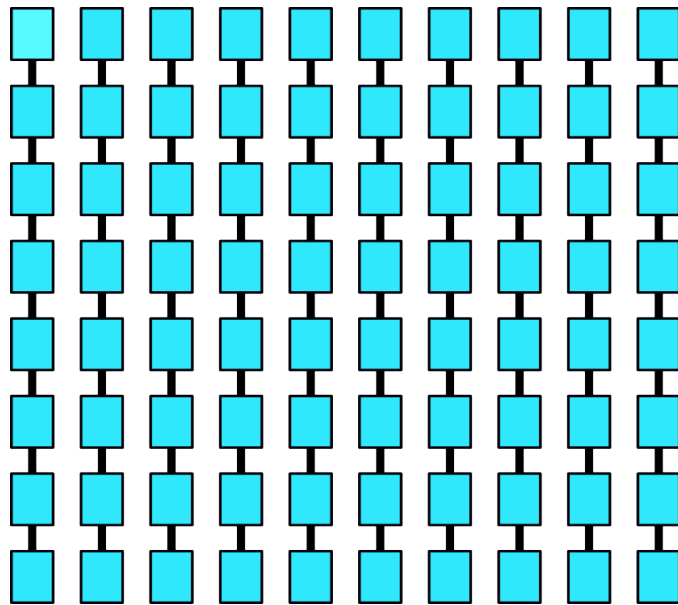


$$x_{i,j}^{r*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

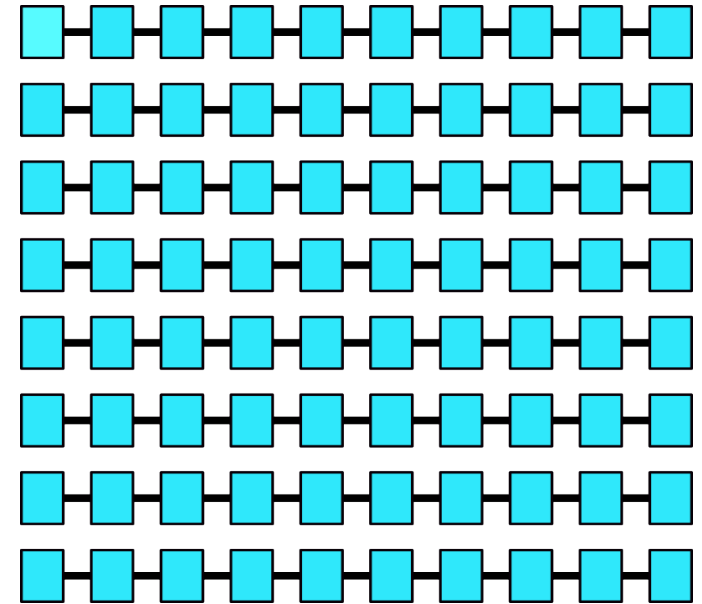
Super-gradient:

$$\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$$



$$x_{i,j}^{c*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling



$$x_{i,j}^{r*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

Super-gradient: $\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$

Let $E_{i,j}^c(s) = E_{i,j}^r(s)$ for all $s \in \mathcal{X}_{i,j}$. Then:

- 1) All super-gradients are equal to 0
- 2) All super-gradients are non-zero
- 3) There is always a super-gradient equal to 0
- 4) There might be a zero super-gradient, however, it is not guaranteed
- 5) No idea
- 6) None of 1-4 is correct

$$x_{i,j}^{c*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

$$x_{i,j}^{r*} \in \arg \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

Super-gradient: $\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$

$$E_{i,j}^c(s) = E_{i,j}^r(s)$$

$$C + \lambda' = R - \lambda'$$

$$x_{i,j}^{c*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

$$x_{i,j}^{r*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

Super-gradient: $\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$

$$E_{i,j}^c(s) = E_{i,j}^r(s)$$

$$C + \lambda' = R - \lambda' \quad \Rightarrow \quad \lambda' = \frac{R - C}{2} = \frac{(E^r + \lambda) - (E^c - \lambda)}{2}$$

$$x_{i,j}^{c*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

$$x_{i,j}^{r*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

Super-gradient: $\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$

$$E_{i,j}^c(s) = E_{i,j}^r(s)$$

$$C + \lambda' = R - \lambda' \quad \Rightarrow \quad \lambda' = \frac{R - C}{2} = \frac{(E^r + \lambda) - (E^c - \lambda)}{2}$$

$$\lambda' = \lambda + \frac{E^r - E^c}{2}$$

$$x_{i,j}^{c*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

$$x_{i,j}^{r*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

Super-gradient: $\frac{\partial U}{\partial \lambda_v} = \delta(x^{c*})_v - \delta(x^{r*})_v$

$$E_{i,j}^c(s) = E_{i,j}^r(s)$$

$$C + \lambda' = R - \lambda' \quad \Rightarrow \quad \lambda' = \frac{R - C}{2} = \frac{(E^r + \lambda) - (E^c - \lambda)}{2}$$

$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

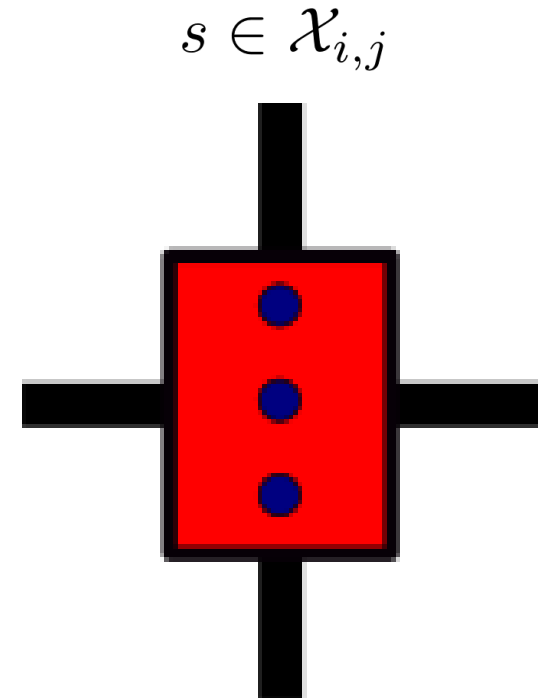
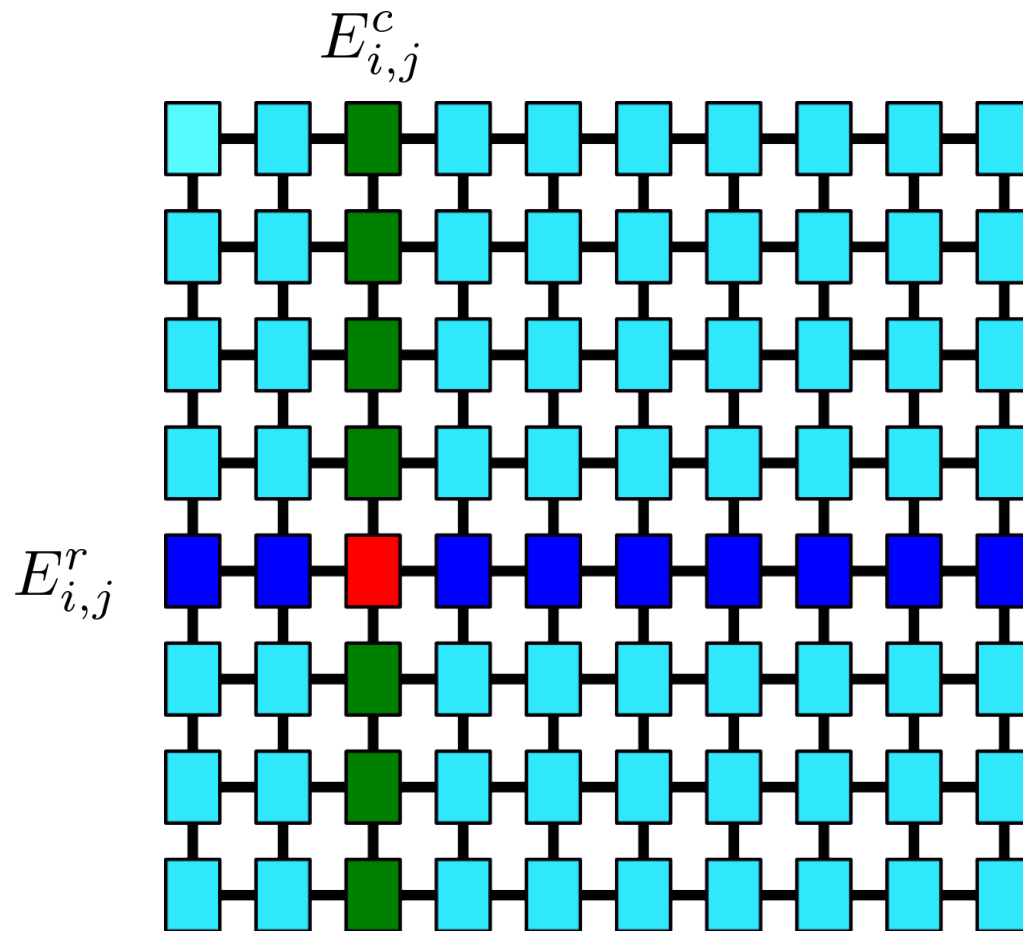
$$x_{i,j}^{c*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^c(s)$$

Column-wise labeling

$$x_{i,j}^{r*} = \min_{s \in \mathcal{X}_{i,j}} E_{i,j}^r(s)$$

Row-wise labeling

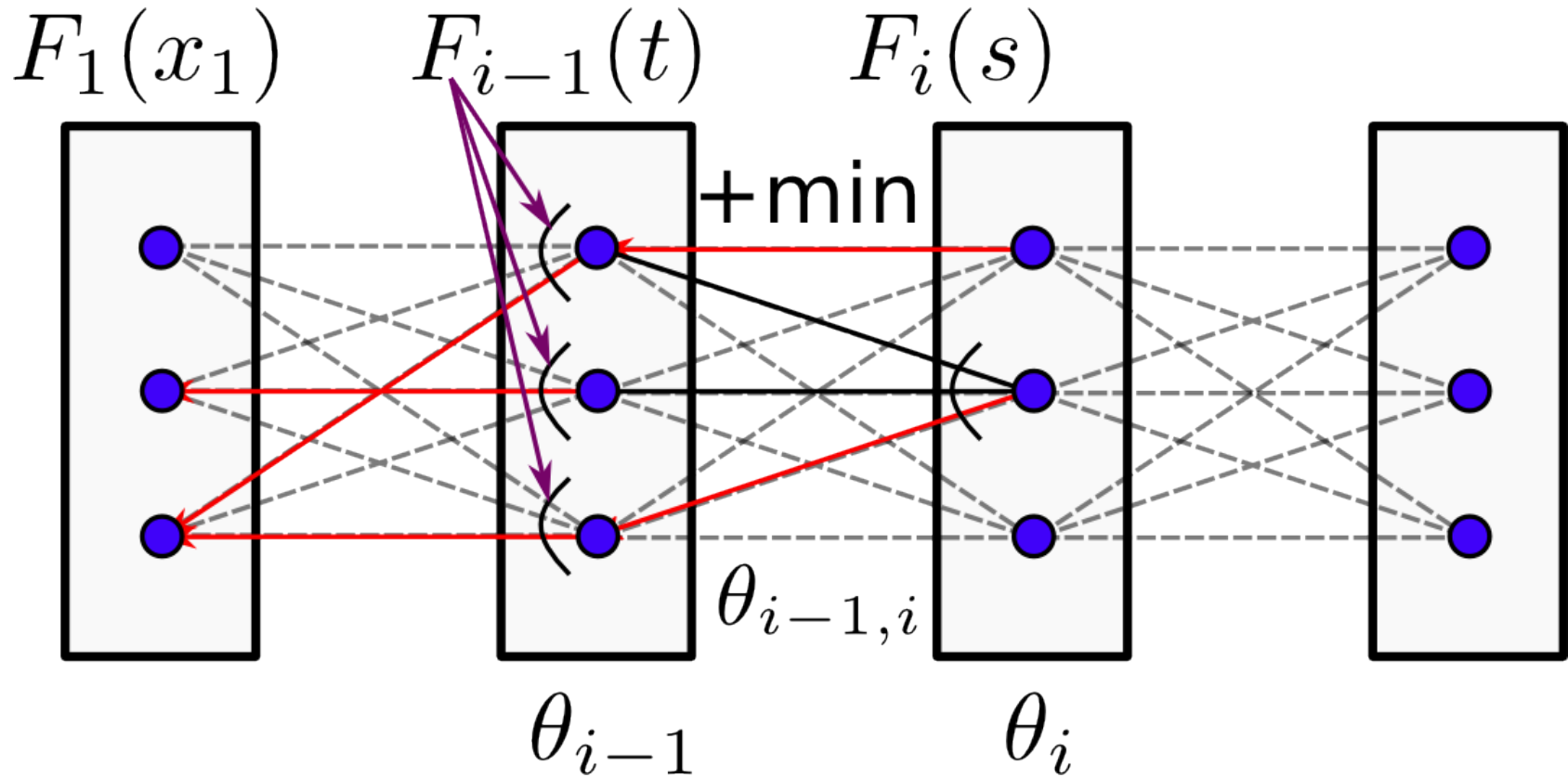
Principal Tree-Reweighted Block-Coordinate Ascent



$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

Computation of Min-Marginals

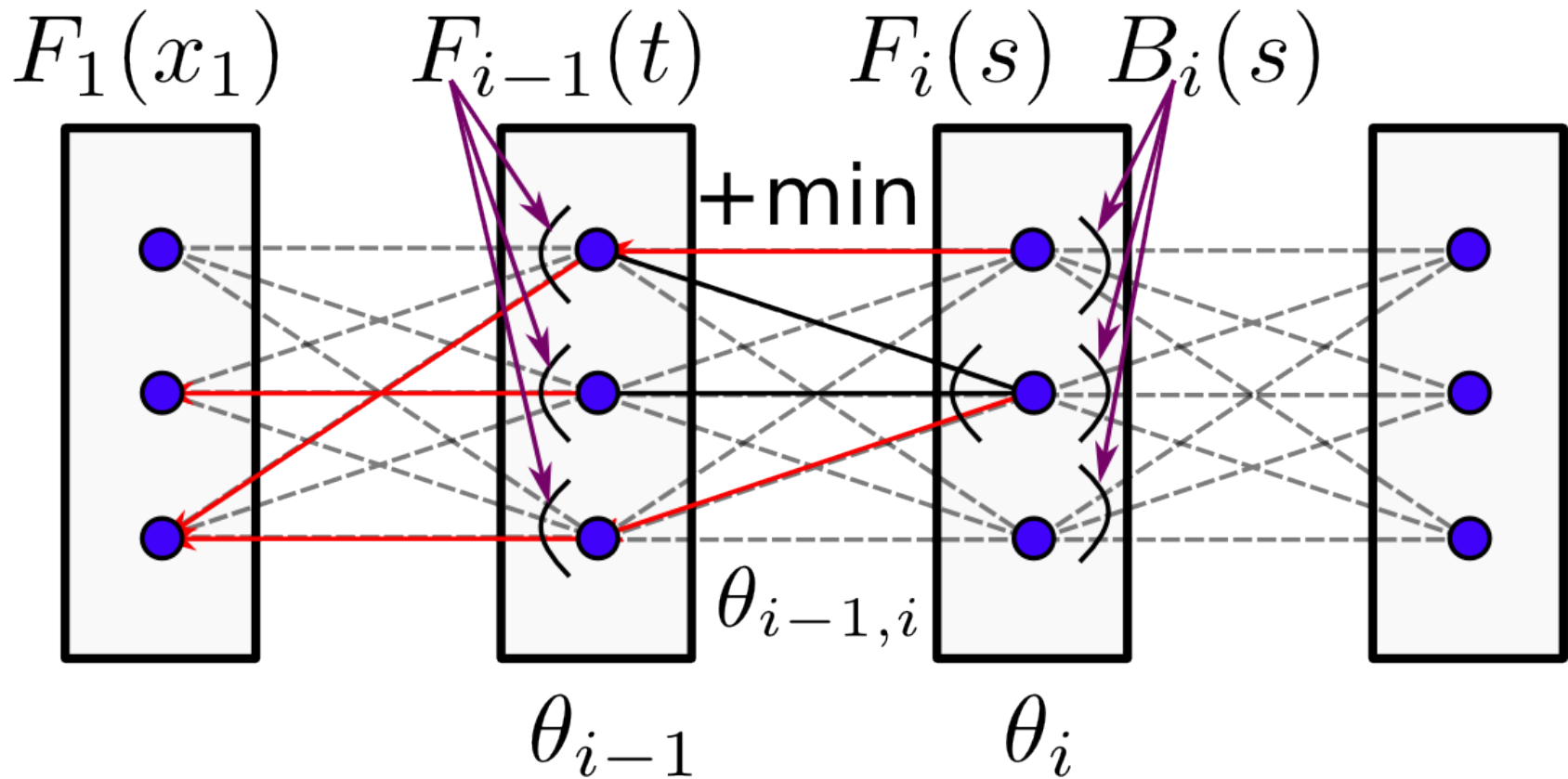
Compute the shortest paths from the left and right:



$$F_i(s) := \min_{t \in \mathcal{X}_{i-1}} \theta_{i-1}(t) + F_{i-1}(t) + \theta_{(i,i-1)}(s, t)$$

Computation of Min-Marginals

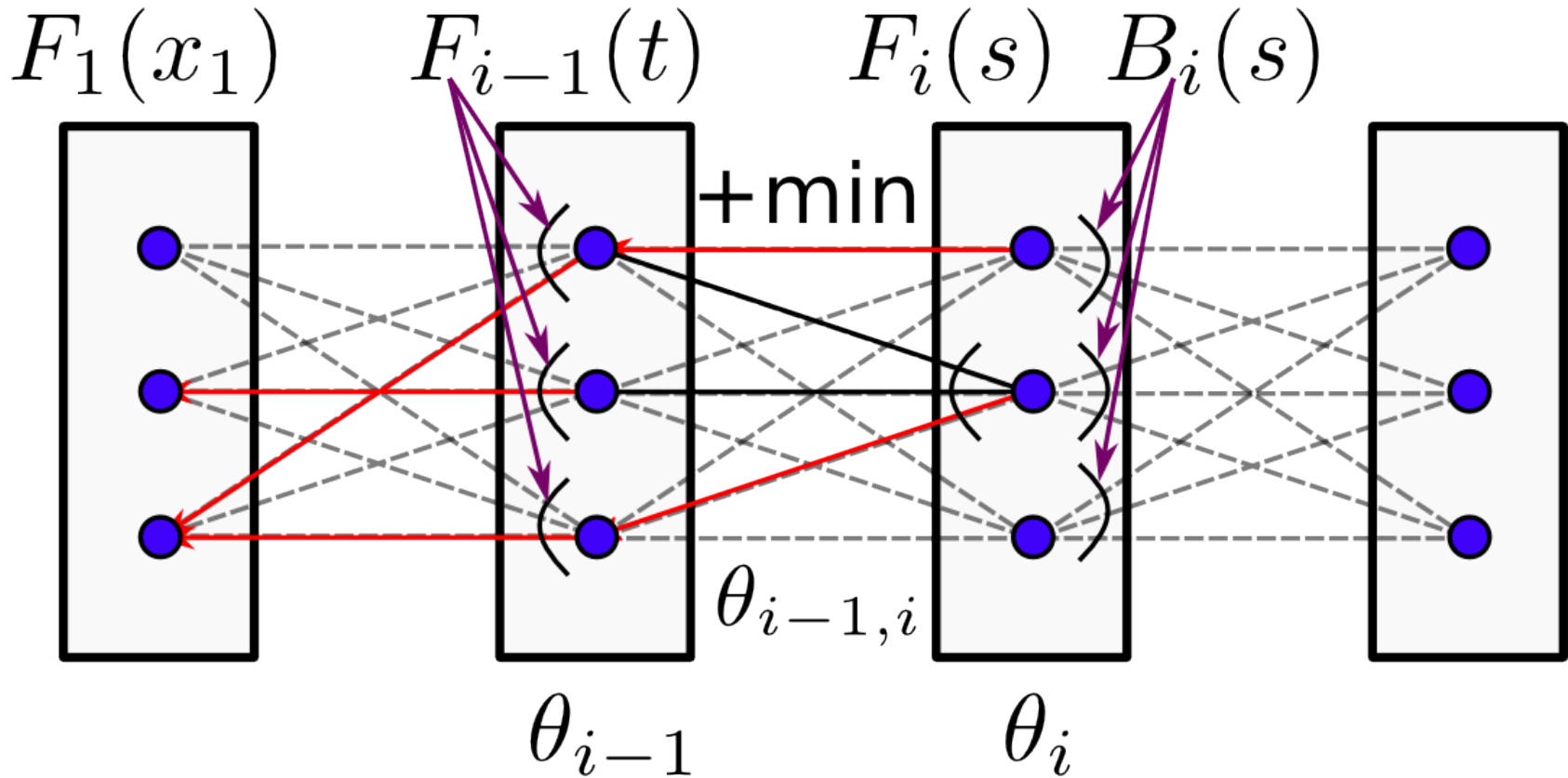
Compute the shortest paths from the left and right:



$$E_i(s) = F_i(s) + \theta_i(s) + B_i(s)$$

Computation of Min-Marginals

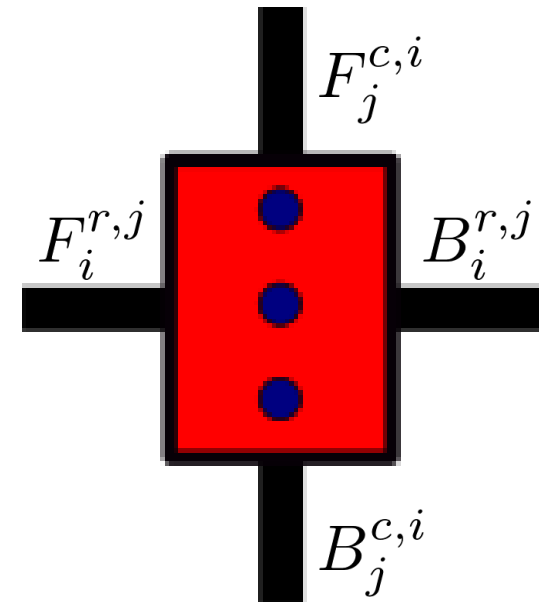
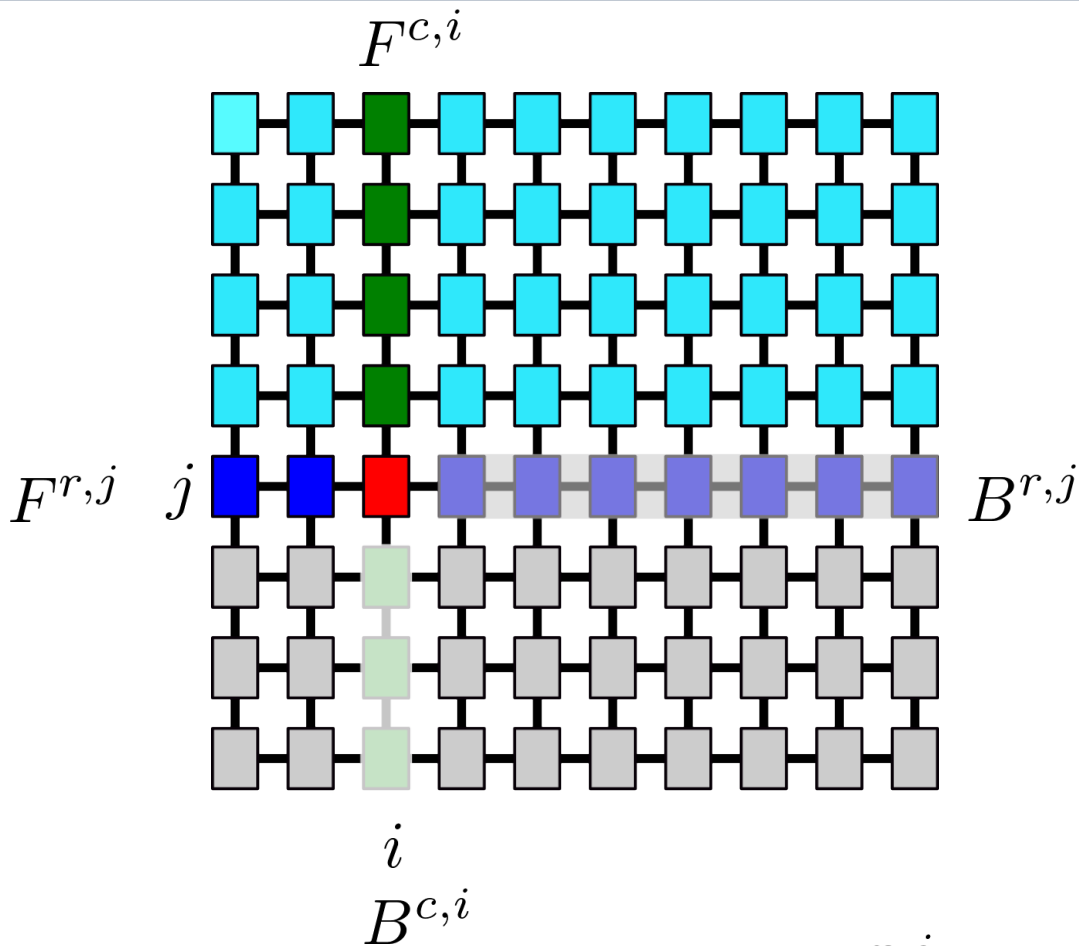
If we change θ_i , then $F_{i'}$ for $i' \leq i$ and $B_{i'}$ for $i' \geq i$ remain valid



$$E_i(s) = F_i(s) + \theta_i(s) + B_i(s)$$

$$F_i(s) := \min_{t \in \mathcal{X}_{i-1}} \theta_{i-1}(t) + F_{i-1}(t) + \theta_{(i,i-1)}(s, t)$$

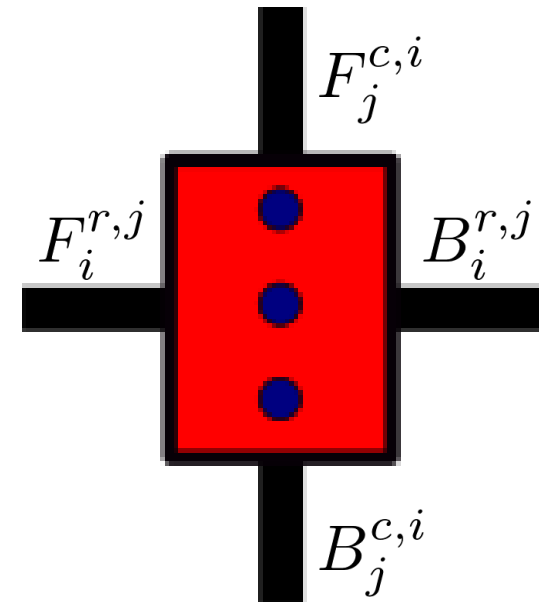
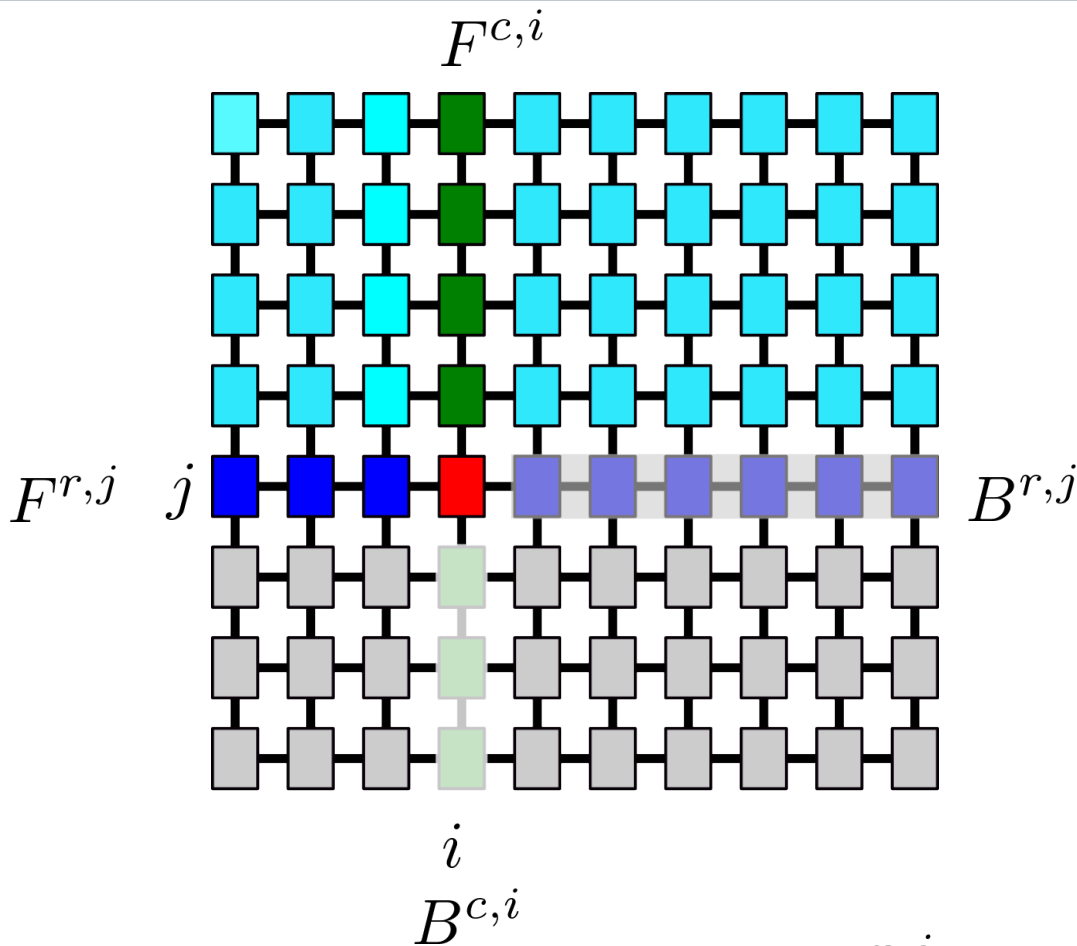
Implementation of Tree-Reweighted Block-Coordinate Ascent



$$E_i^{r,j}(s) = F_i^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

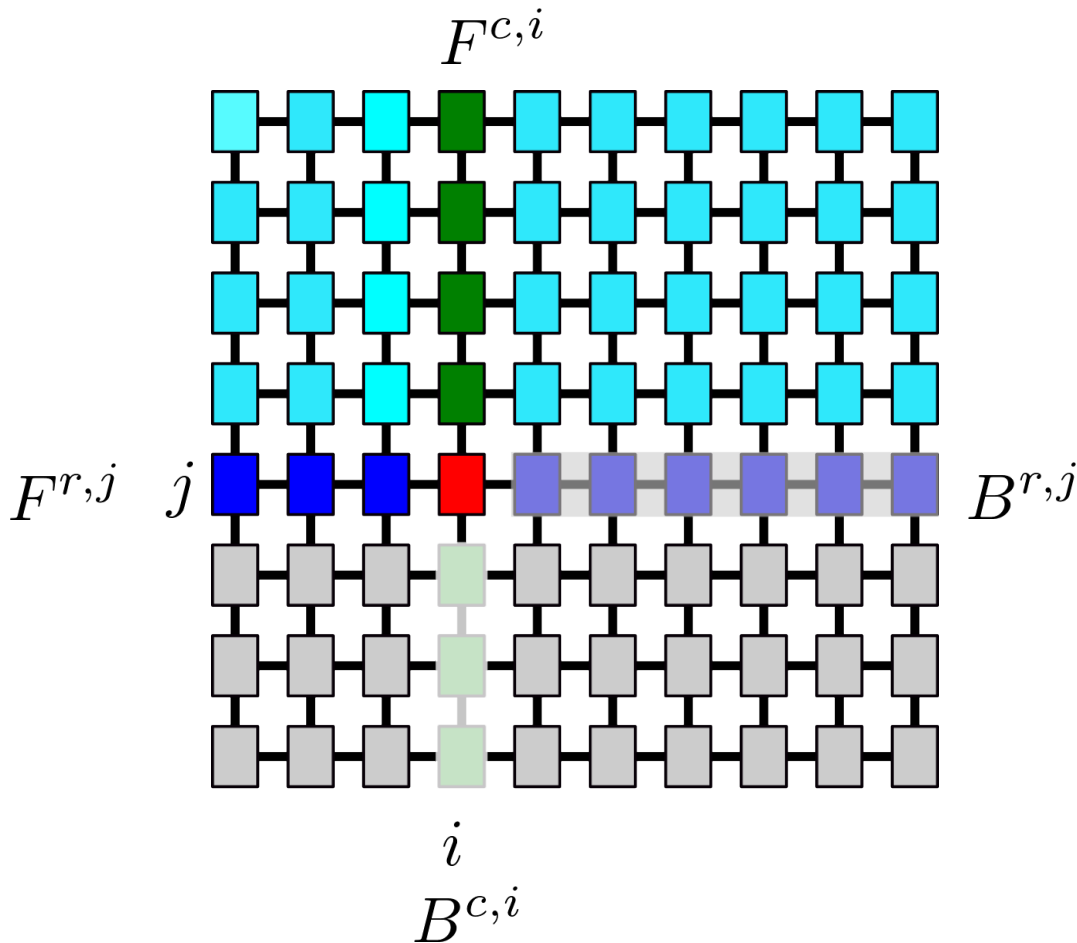
Implementation of Tree-Reweighted Block-Coordinate Ascent



$$E_i^{r,j}(s) = F_i^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

Implementation of Tree-Reweighted Block-Coordinate Ascent

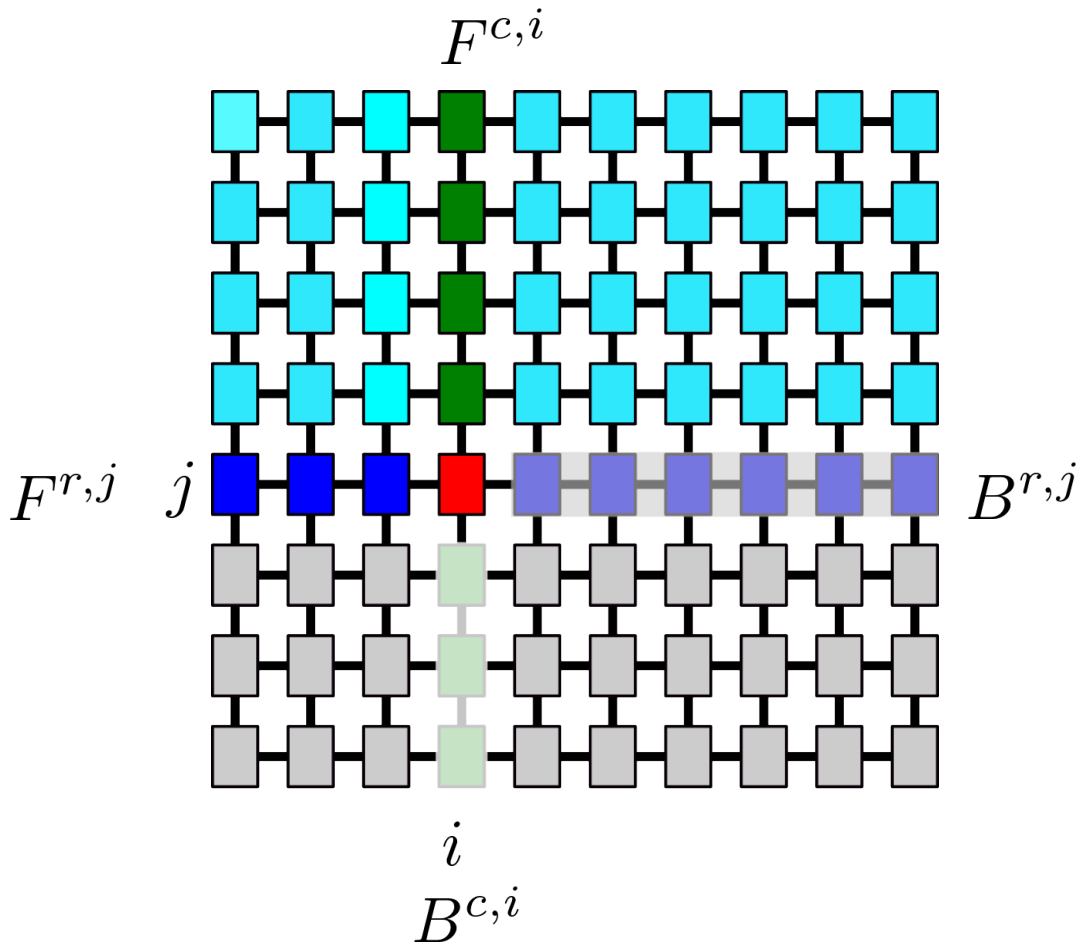


Requirement:

$$\lambda, B^r, B^c$$

Result: valid F^r, F^c

Implementation of Tree-Reweighted Block-Coordinate Ascent



Requirement:

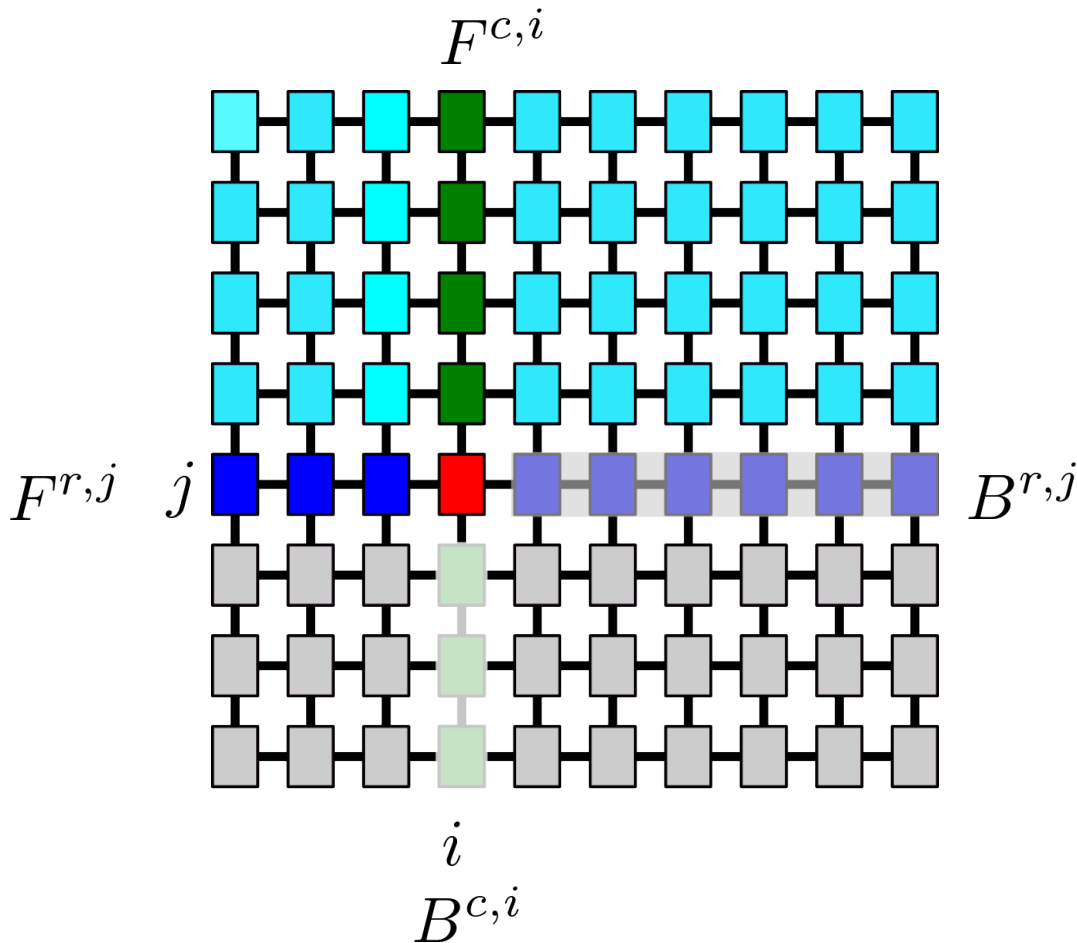
$$\lambda, B^r, B^c$$

Result: valid F^r, F^c

Therefore:

- invert order and $B \leftarrow F$
- iterate

Implementation of Tree-Reweighted Block-Coordinate Ascent



Requirement:

$$\lambda, B^r, B^c$$

Result: valid F^r, F^c

Therefore:

- invert order and $B \leftarrow F$
- iterate

Initialize:

- compute B
- or
- set $B := 0$

TRW-S Algorithm (V. Kolmogorov, 2005)

Summary: Properties of TRW-S Algorithm

- block-coordinate ascent. Each block = variables, associated with one node
- monotonously non-decreasing dual values
- each iteration has the same complexity as computing a super-gradient
- each iteration sequentially updates all variables
- primal integer solution can be approximated by rounding
- *converges to arc-consistency:*

- *find an optimal reparametrization for each row and column separately*

- *compute new potentials for the whole problem:* $\theta_v^\phi := \theta_v^{c,\phi} + \theta_v^{r,\phi}$

$$\theta_{uv}^\phi := \theta_{uv}^{c,\phi} \text{ or } \theta_{uv}^\phi := \theta_{uv}^{r,\phi}$$

- *these potentials have a non-empty kernel (arc-consistent subset)*

Summary: Properties of TRW-S Algorithm

- block-coordinate ascent. Each block = variables, associated with one node
- monotonously non-decreasing dual values
- each iteration has the same complexity as computing a super-gradient
- each iteration sequentially updates all variables
- primal integer solution can be approximated by rounding
- *converges to arc-consistency:*

- *find an optimal reparametrization for each row and column separately*

- *compute new potentials for the whole problem:* $\theta_v^\phi := \theta_v^{c,\phi} + \theta_v^{r,\phi}$

$$\theta_{uv}^\phi := \theta_{uv}^{c,\phi} \text{ or } \theta_{uv}^\phi := \theta_{uv}^{r,\phi}$$

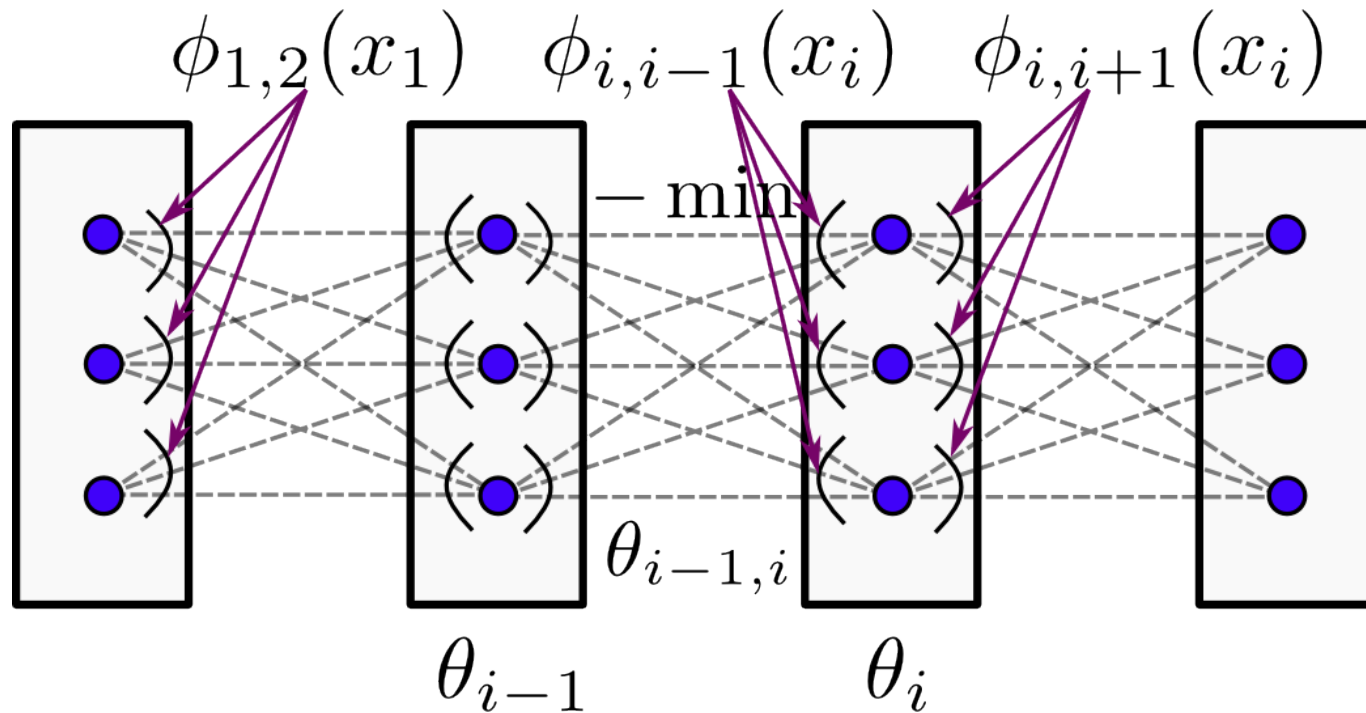
- *these potentials have a non-empty kernel (arc-consistent subset)*

Exercise: Check that

- θ^ϕ is indeed, a reparametrization of θ
- if strong tree-agreement holds, $\text{kern}(\theta^\phi)$ is strictly arc-consistent

Dynamic Programming as Dual Solver

- find an optimal reparametrization for each row and column separately



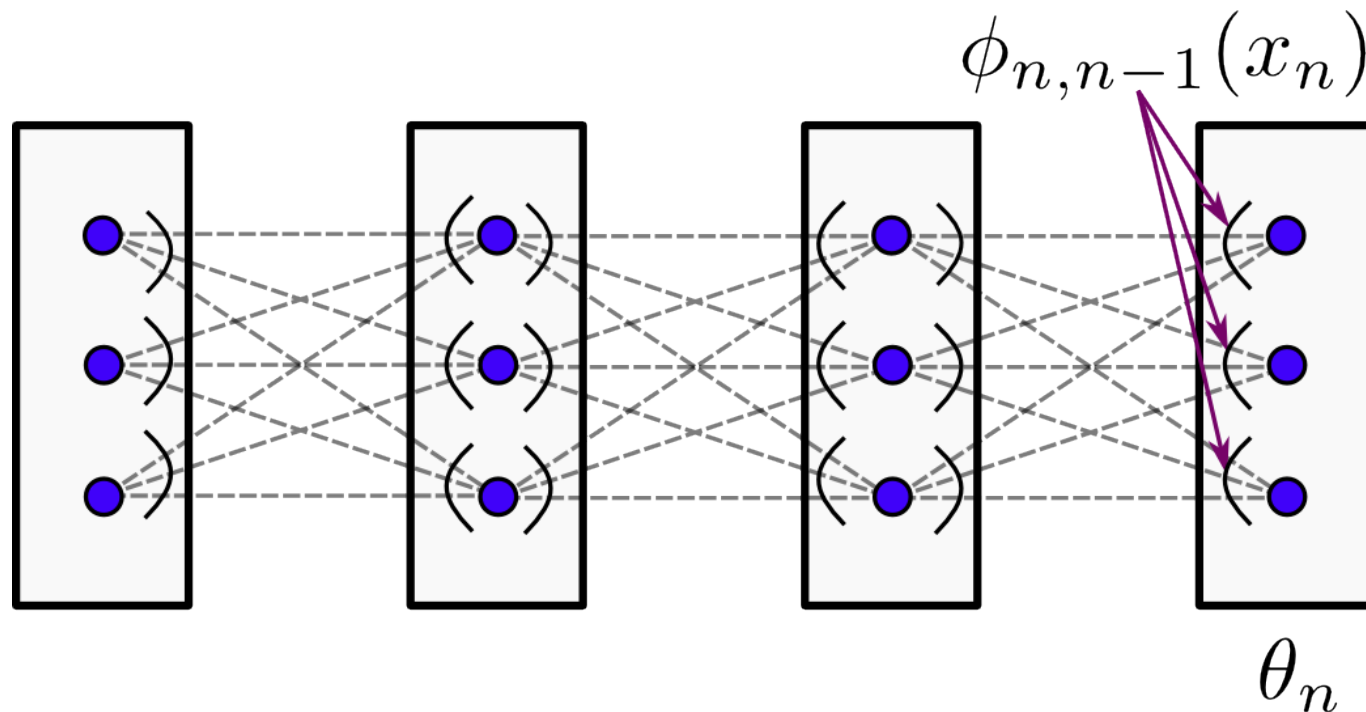
Init: $\phi := 0$. Repeat for $i = 1 \dots n$:

$$\phi_{i,i-1}(x_i) := - \min_{x_{i-1}} \theta_{i-1,i}^{\phi}(x_i) = - \min_{x_{i-1}} \theta_{i-1,i}(x_i) - \phi_{i,i-1}(x_i)$$

$$\phi_{i,i+1}(x_i) := \theta_i^{\phi}(x_i) = \theta_i(x_i) - \phi_{i,i-1}(x_i)$$

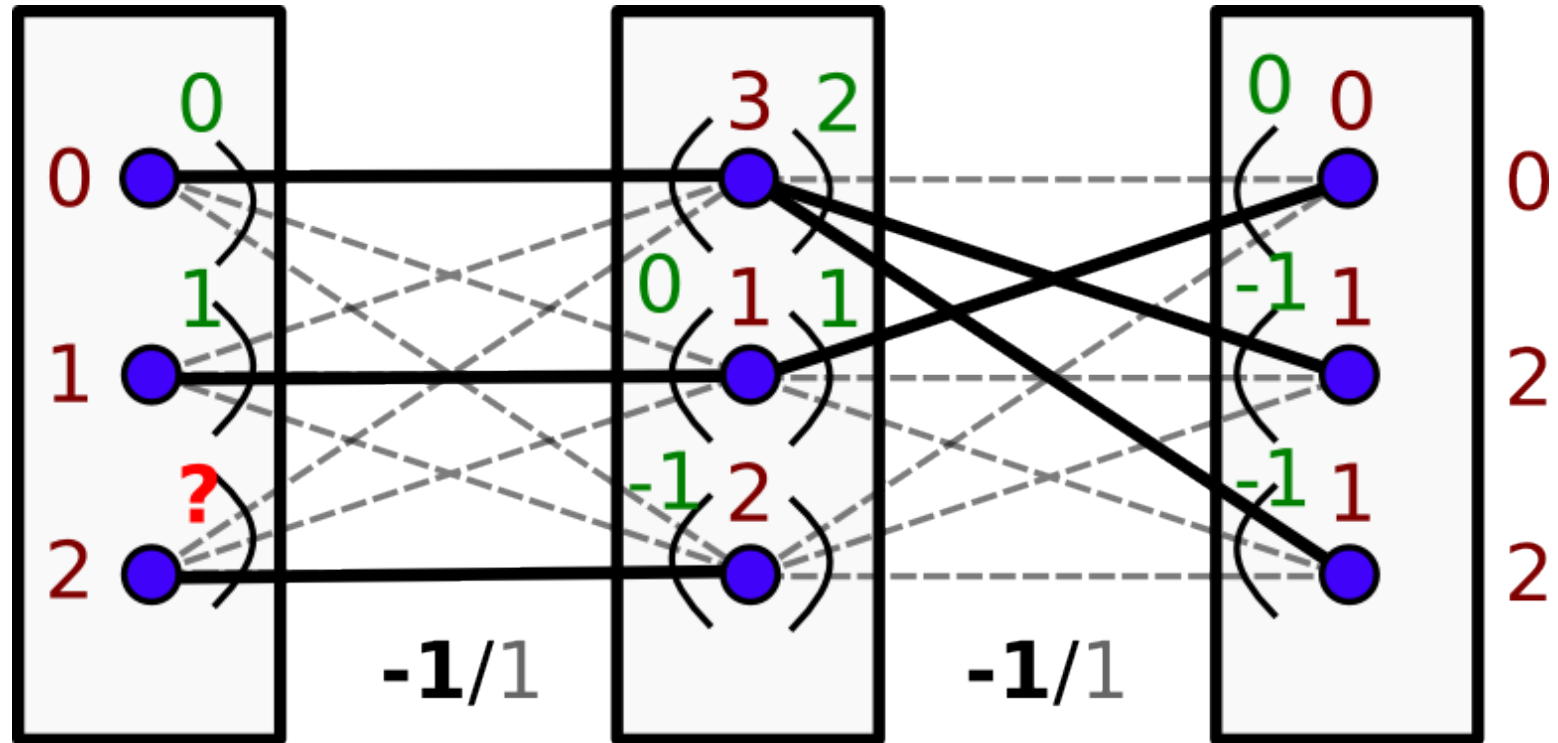
Dynamic Programming as Dual Solver

- find an optimal reparametrization for each row and column separately



$$\min_x E(x) = \theta_i^\phi(x_n) \equiv \theta_n(x_n) - \phi_{n,n-1}(x_n)$$

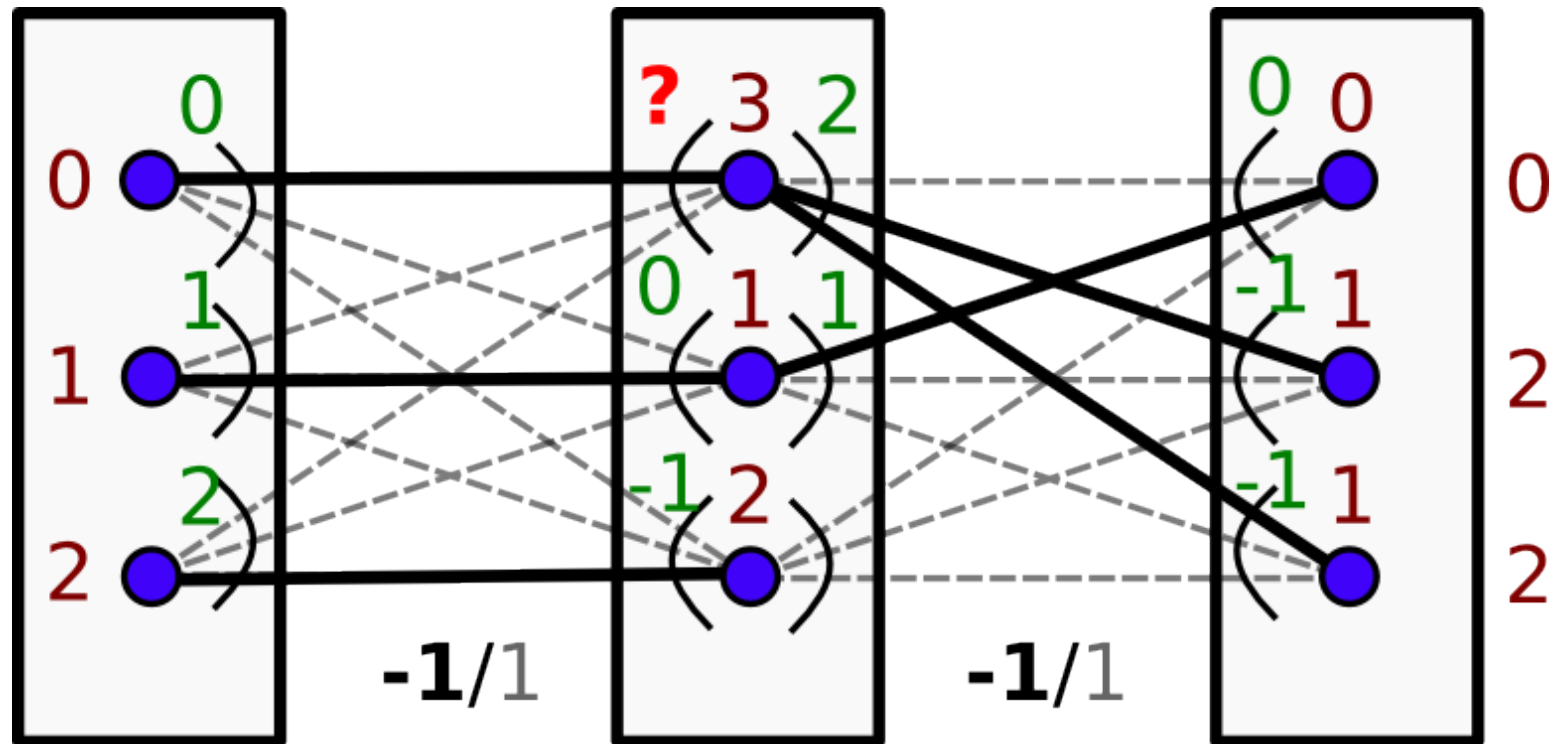
Dynamic Programming as Dual Solver: Example



$$\phi_{i,i+1}(x_i) := \theta_i^\phi(x_i) \equiv \theta_i(x_i) - \phi_{i,i-1}(x_i)$$

- 1) 1
- 2) 2
- 3) 3
- 4) -2
- 5) none is correct
- 6) no idea

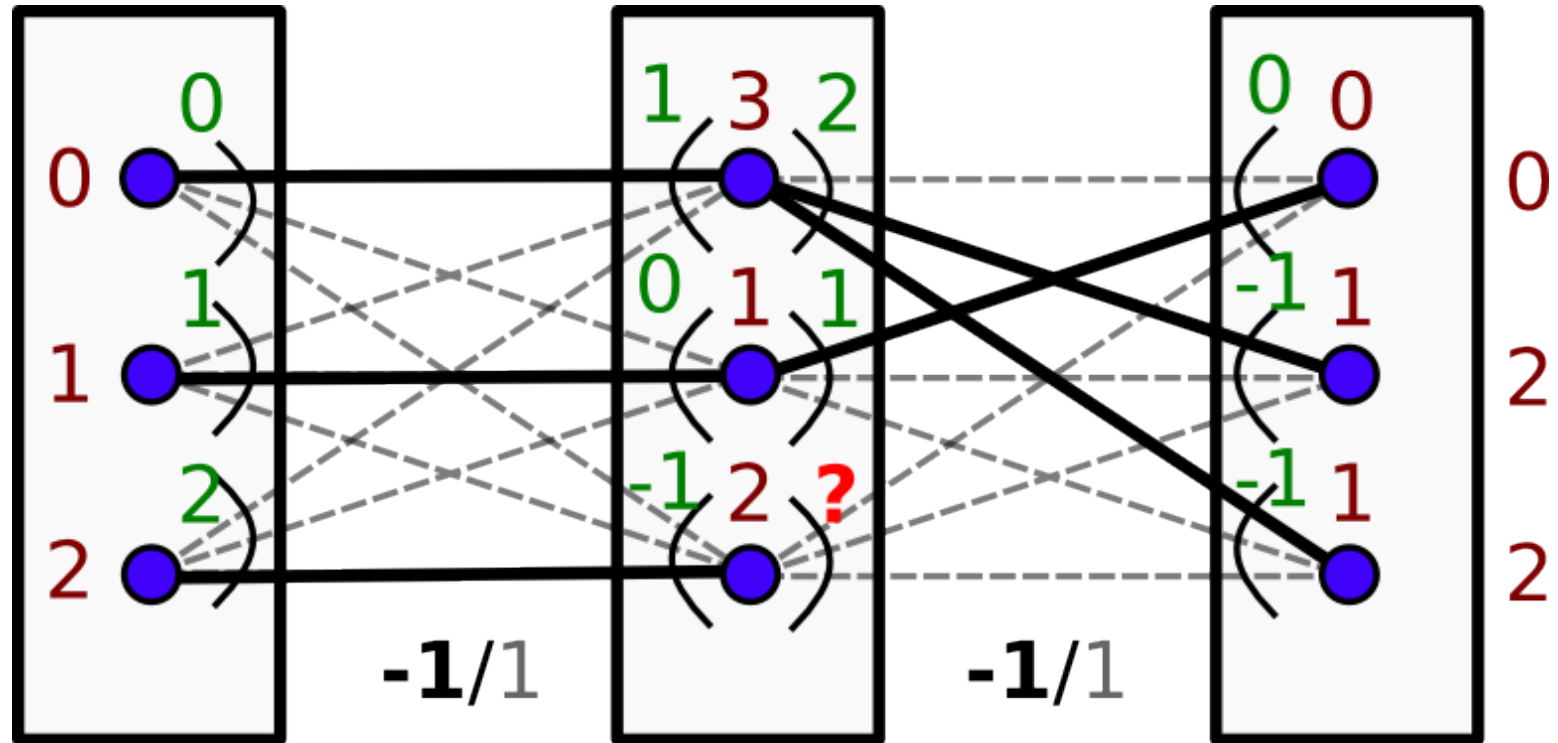
Dynamic Programming as Dual Solver: Example



$$\phi_{i,i-1}(x_i) := -\min_{x_{i-1}} \theta_{i-1,i}^\phi(x_i) \equiv -\min_{x_{i-1}} \theta_{i-1,i}(x_i) - \phi_{i,i-1}(x_i)$$

- | | |
|-------|--------------------|
| 1) 1 | 4) -2 |
| 2) 2 | 5) none is correct |
| 3) -1 | 6) no idea |

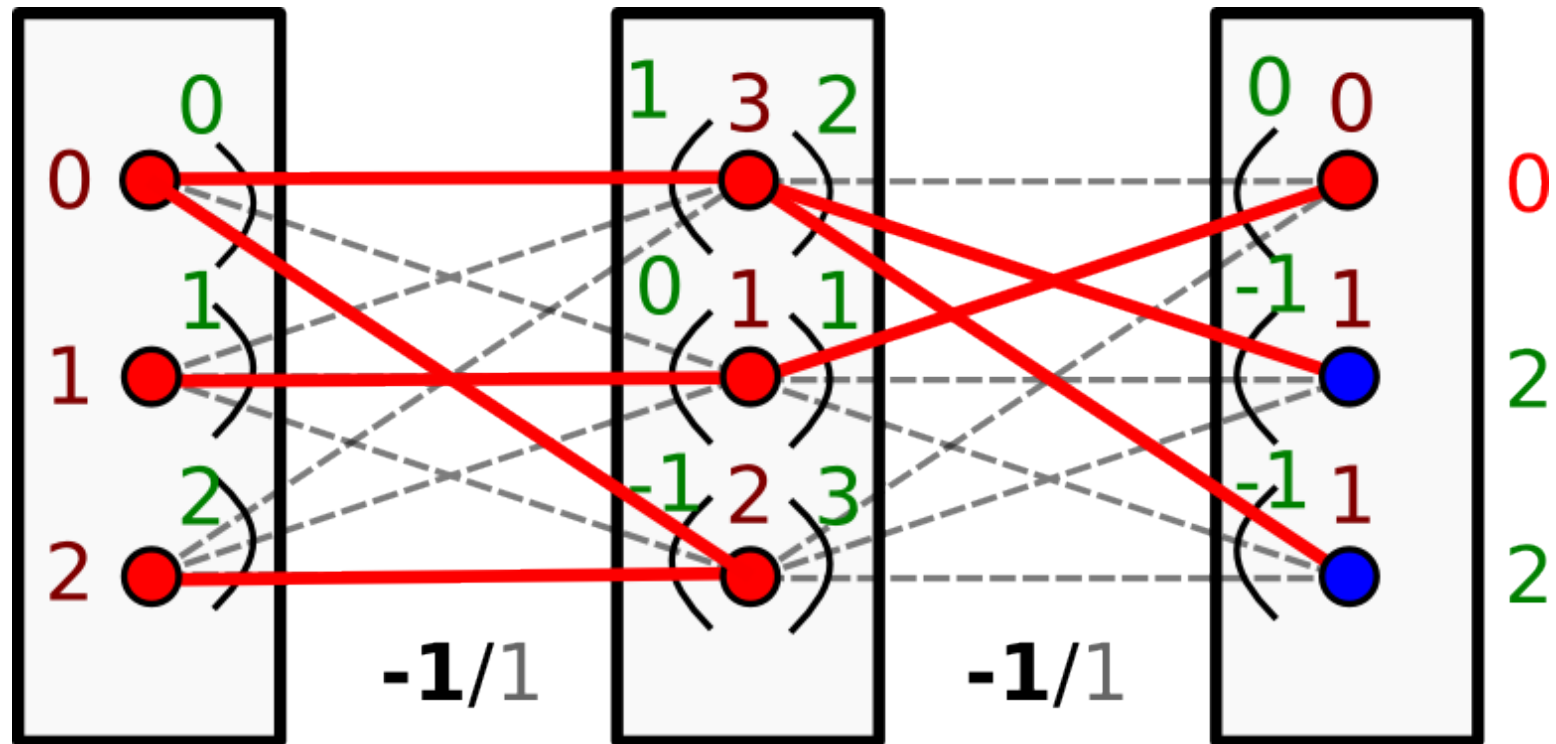
Dynamic Programming as Dual Solver: Example



$$\phi_{i,i+1}(x_i) := \theta_i^\phi(x_i) \equiv \theta_i(x_i) - \phi_{i,i-1}(x_i)$$

- 1) 1
- 2) 2
- 3) 3
- 4) -3
- 5) none is correct
- 6) no idea

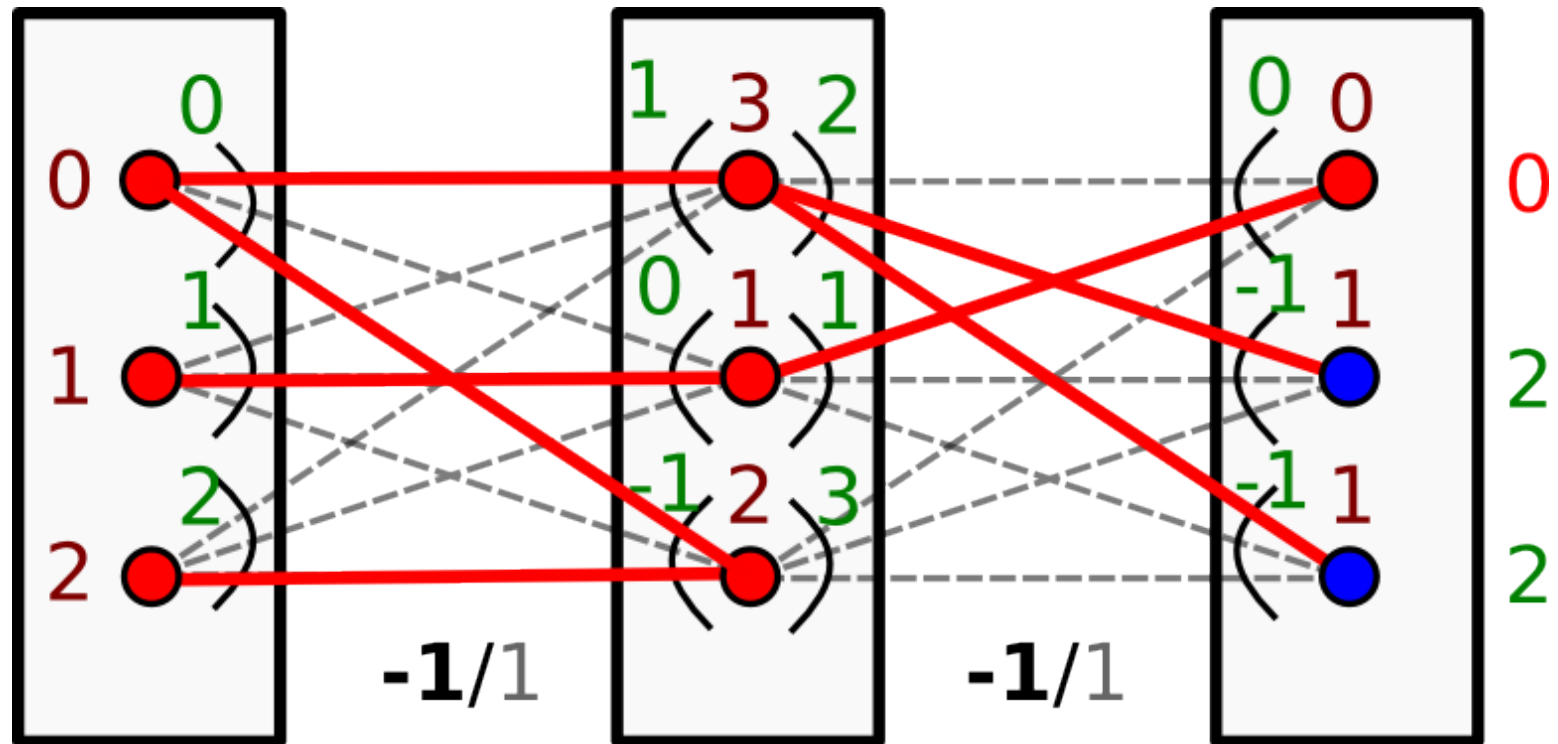
Dynamic Programming as Dual Solver: Example



This reparametrization:

- 1) arc-consistent
- 2) has a non-empty kernel
- 3) strictly arc-consistent
- 4) has an empty kernel
- 5) none of the above
- 6) no idea

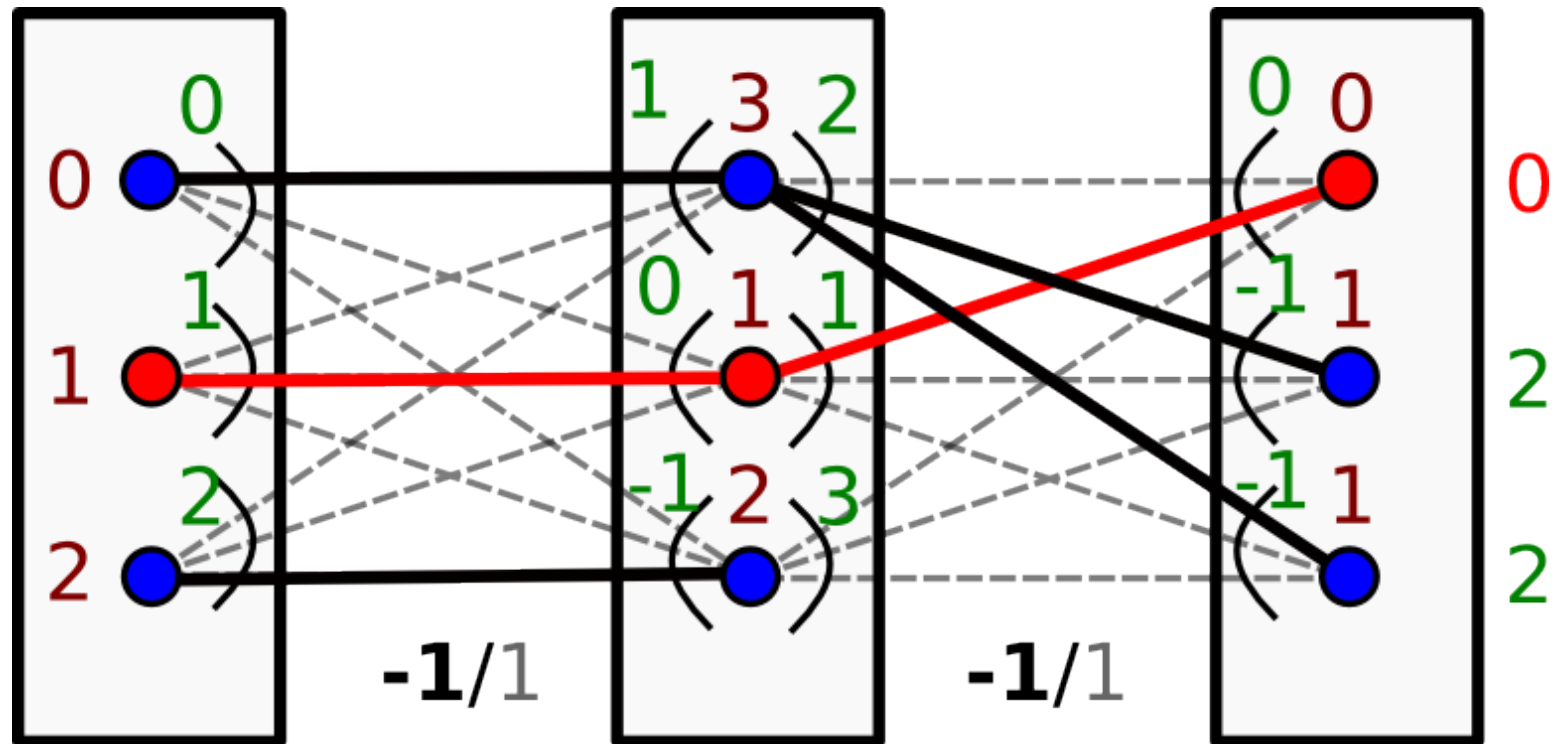
Dynamic Programming as Dual Solver: Example



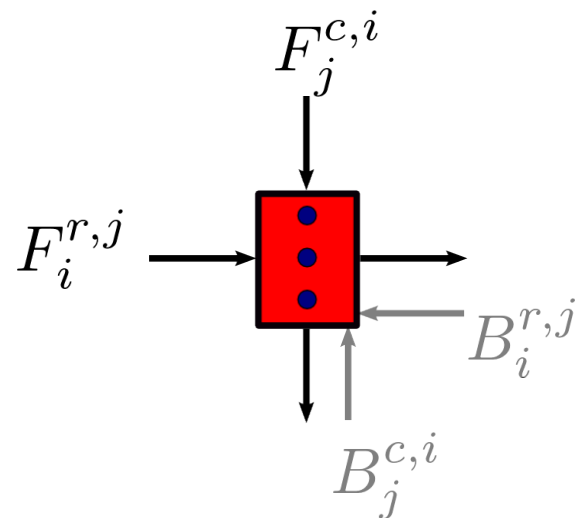
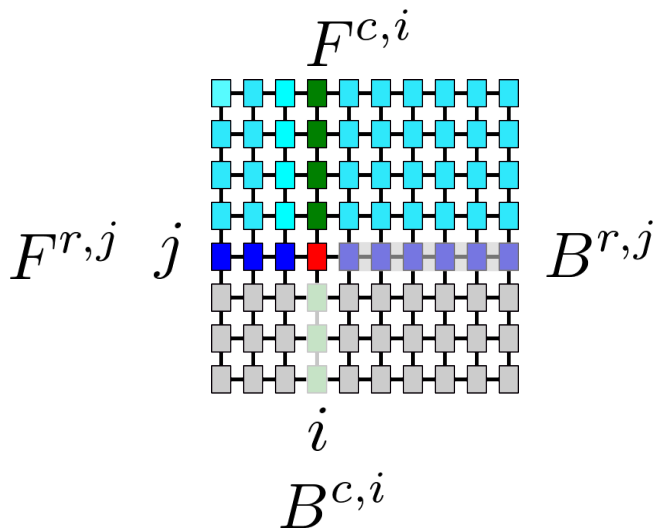
This reparametrization:

- 1) optimal
- 2) not optimal
- 3) no idea

Dynamic Programming as Dual Solver: Example



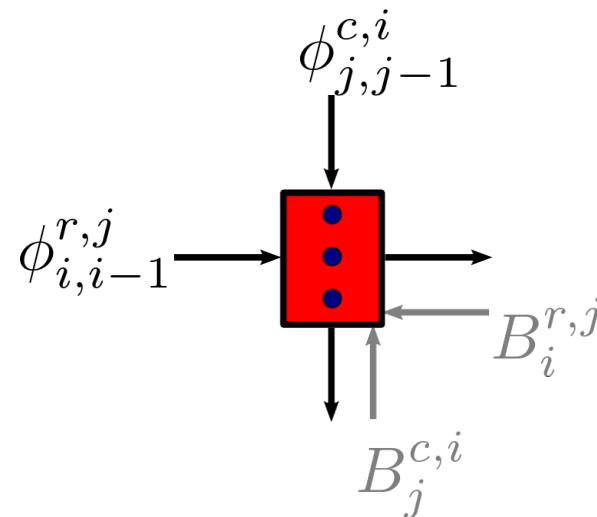
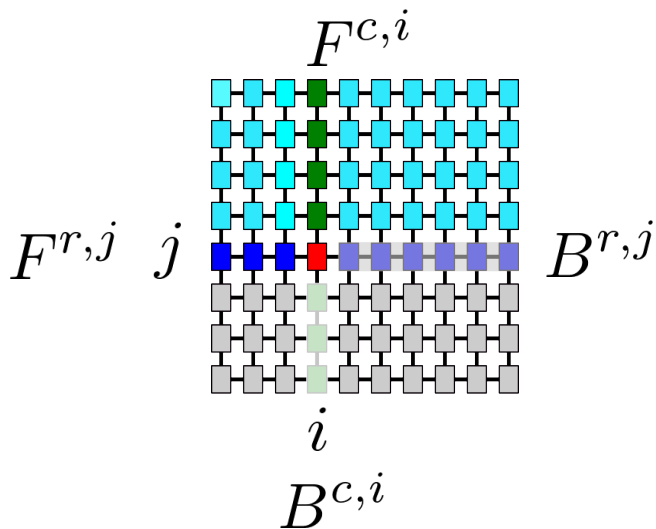
TRW-S: Reparametrization-Based Representation



$$E_i^{r,j}(s) = F_i^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

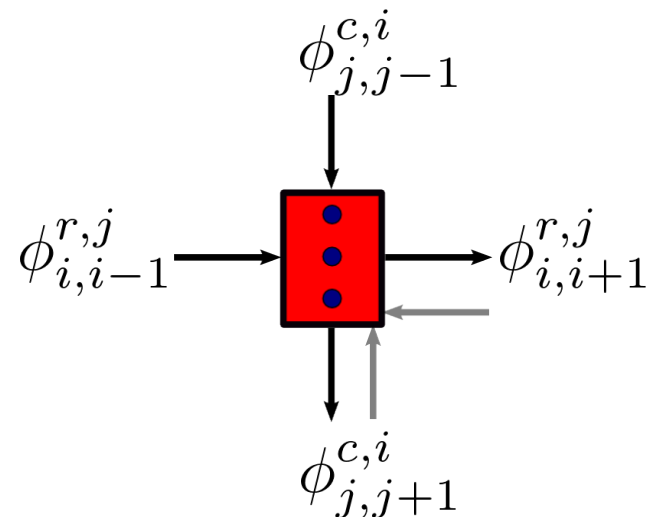
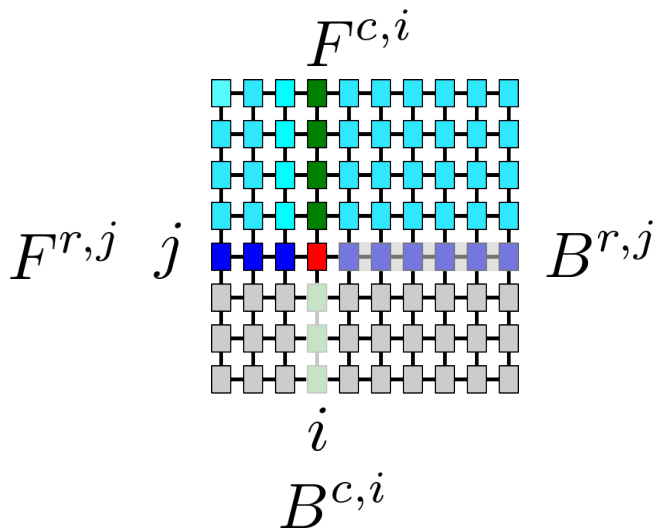
$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

TRW-S: Reparametrization-Based Representation



$$E_i^{r,j}(s) = \phi_{i,i-1}^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

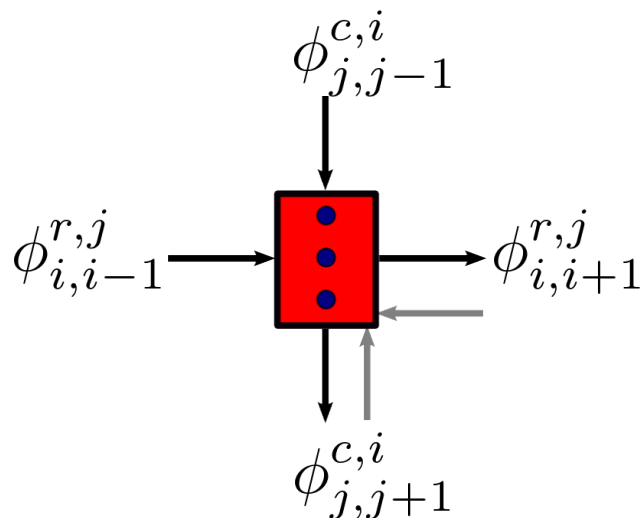
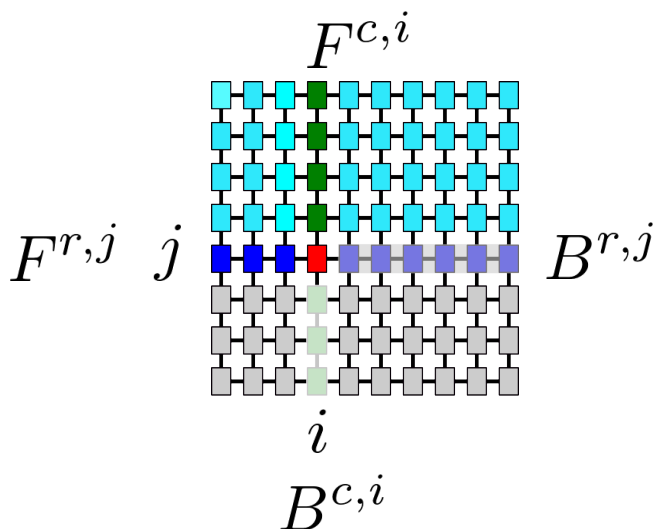
$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$



$$E_i^{r,j}(s) = \phi_{i,i-1}^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

$$\lambda'_{i,j}(s) = \lambda_{i,j}(s) + \frac{E_{i,j}^r(s) - E_{i,j}^c(s)}{2}$$

TRW-S: Reparametrization-Based Representation

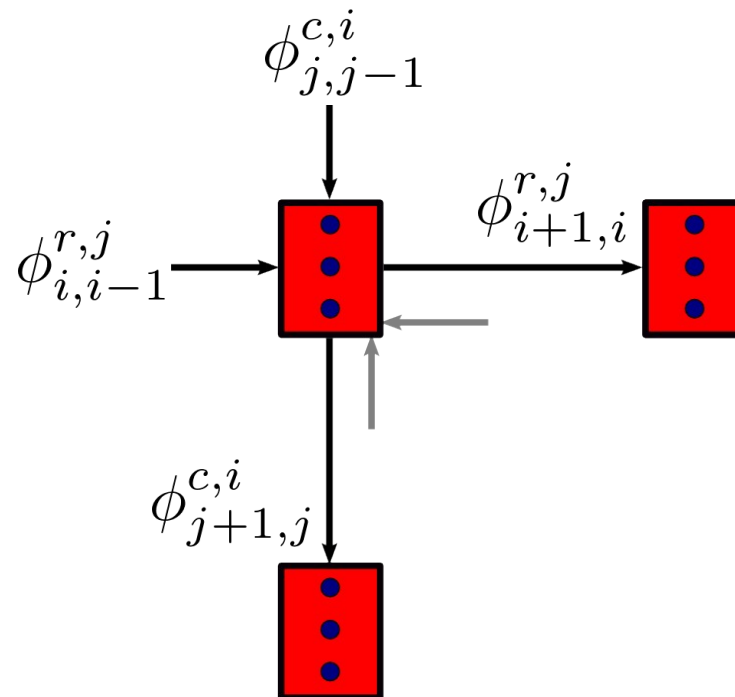
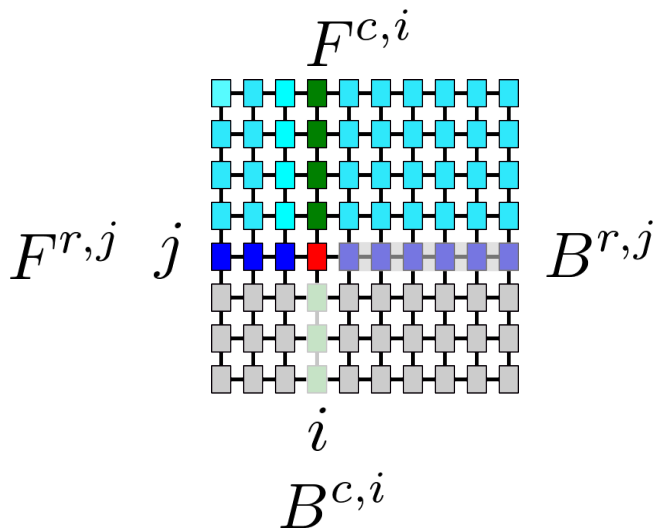


No need to store $\phi_{j,j+1}^{c,i}$ and $\phi_{i,i+1}^{r,j}$

Directly compute $\phi_{j+1,j}^{c,i}$ and $\phi_{i+1,i}^{r,j}$

$$E_i^{r,j}(s) = \phi_{i,i-1}^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

TRW-S: Reparametrization-Based Representation

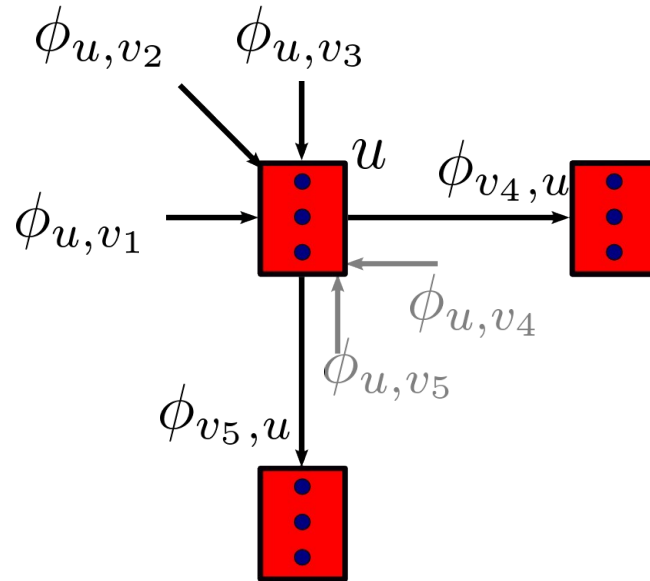
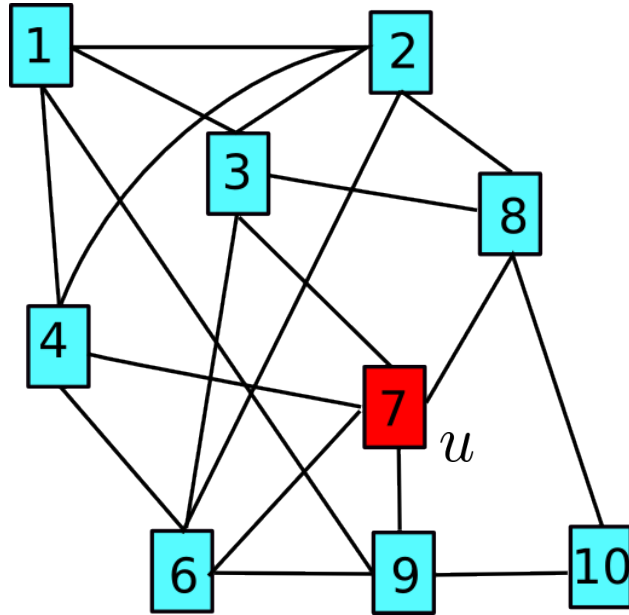


No need to store $\phi_{j,j+1}^{c,i}$ and $\phi_{i,i+1}^{r,j}$

Directly compute $\phi_{j+1,j}^{c,i}$ and $\phi_{i+1,i}^{r,j}$

$$E_i^{r,j}(s) = \phi_{i,i-1}^{r,j}(s) + \theta_i^{r,j}(s) + B_i^{r,j}(s)$$

For general graphs:



- 1) Introduce order on vertexes
- 2) direct edges (u,v) , $u < v$
- 3) initialize $\phi = 0$
- 4) run an iteration
- 5) reverse the vertex order
- 6) repeat 4-5

$$\phi_{v,u}(x_v) := - \min_{x_u} \left(\frac{1}{\max\{n_{in}, n_{out}\}} \theta_v^\phi + \phi_{u,v} + \theta_{u,v}^\phi(x_u, x_v) \right)$$

Your conclusions are welcome!